

Product Analyst Intern - TAKE-HOME **ASSIGNMENT**

Task 1 — What doesn't match?

I found 5 mismatches between the API documentation and captured responses:

1. The status field contains an undocumented value

The documentation says that `status` can only be `pending`, `shipped`, `delivered`, or `cancelled`. However, in `orders_page1.json`, the `status` field for `ord_1003` is `"refunded"`.

This matters because an integration validating the `status` field against the documented values could reject or mishandle this order.

2. The monetary fields do not always follow the documented format

The documentation says that `subtotal`, `tax`, `shipping`, and `total` are integers representing the smallest currency unit. However, in `orders_page2.json`, the `subtotal`, `tax`, `shipping`, and `total` fields for `ord_1006` are returned as `44.0`, `3.63`, `5.99`, and `53.62`.

This matters because a client expecting integer minor-unit values could interpret these amounts incorrectly.

3. customer.email is documented as always being present

The documentation says that `customer.email` is always present. However, in `orders_page2.json`, the `customer.email` field for `ord_1005` is `null`.

This could cause problems for integrations that assume the field will always contain a string.

4. A missing order returns 200 instead of 404

For `GET /v1/orders/ord_9999`, the documentation says that a nonexistent order should return `404`. The captured response in `order_ord_9999.json` instead returns `200` with `"order": null`.

This makes it harder for a client to distinguish between a successful request and an order that does not exist.

5. The pagination information is inconsistent

In `orders_page1.json`, the pagination fields show `has_more: false` while `next_cursor` contains `cur_8f2a19bd`. The documentation says `has_more` should determine whether another page is requested. A second captured response, `orders_page2.json`, uses that cursor and contains `ord_1005` and `ord_1006`.

This means a client following `has_more` could stop after the first page and silently miss additional orders.

Most serious mismatch:

The pagination issue is the most serious because it can cause entire orders to be missed without producing an obvious error. The API can return a successful response while the client believes it has received all the data, which can directly affect downstream reporting and reconciliation.

Task 2 — What's the total revenue?

I treated the `total` field as the amount to add for each order and included all six orders, including `ord_1003`, because the data gives no rule for excluding refunded orders.

The first five orders use the documented minor-unit format and add up to \$274.41. For `ord_1006`, the `total` is `53.62` rather than an integer minor-unit value. I treated this as \$53.62 because its `subtotal`, `tax`, and `shipping` values also add up to `53.62`, which suggests these values are being returned as dollar amounts despite the documentation.

Total: \$328.03

This is the total order value based on the available data. If “revenue” is meant to exclude refunded orders or follow a specific accounting treatment, that cannot be determined from the data provided. I would need the rule for how refunded orders should be treated.

Task 3A — Reply to Priya

Subject: Re: Monthly revenue reconciliation

Hi Priya,

I checked the Meridian API orders and found a few inconsistencies that could explain the reconciliation difference.

The six orders add up to **\$328.03**, assuming **53.62** for **ord_1006** is a dollar amount. However, the API documentation says monetary values should be returned as integer minor units, so that order needs clarification. There is also a refunded order, **ord_1003**, but the available data does not tell us whether refunded orders should be included in revenue.

I also found a pagination issue where the first response says there are no more orders even though another response contains two additional orders. This could cause an integration to miss orders.

I would confirm the refund treatment and the intended amount format for **ord_1006**, and check whether the dashboard is applying the same rules.

Regards,
Jaswanth Kumar N

Task 3B — Bug Report

Title: Orders API says there are no more orders when another page exists

Endpoint: **GET /v1/orders**

What to look at:

Check the pagination fields in **orders_page1.json**, especially **has_more** and **next_cursor**.

What happens:

The first response has **has_more: false** but also provides **next_cursor: cur_8f2a19bd**. The second captured response, **orders_page2.json**, uses that cursor and contains two more orders: **ord_1005** and **ord_1006**.

How to reproduce:

1. Request `GET /v1/orders`.
2. Check the `has_more` field.
3. The response says `has_more: false`.
4. Use `cur_8f2a19bd` as `starting_after`.
5. The next response contains `ord_1005` and `ord_1006`.

What should happen:

If more orders are available, `has_more` should be `true` and the response should provide the cursor for the next page. If there are no more orders, `has_more` should be `false`.

Impact:

A client following the API documentation may stop after the first page and miss the additional orders, leading to incomplete data and incorrect reporting.

Suggested fix:

Update the pagination response so that `has_more` correctly shows whether another page of orders is available. Check the same behavior on other pages as well.